CLASE 7 — EVALUACIÓN, MÉTRICAS, VALIDACIÓN Y GENERALIZACIÓN

En la unidad anterior comenzamos a trabajar con modelos concretos de aprendizaje supervisado. Vimos que una misma situación puede ser abordada mediante diferentes estrategias y que cada modelo tiene una forma particular de aprender y de producir una predicción. Sin embargo, quedó planteada una pregunta que resulta mucho más importante que simplemente saber entrenar un modelo: ¿cómo podemos determinar si ese modelo realmente es bueno?

Esta pregunta parece sencilla, pero en Machine Learning tiene bastante profundidad. Supongamos que construimos un modelo capaz de predecir si un cliente comprará un producto y, después de entrenarlo, obtenemos un 95 % de aciertos. A primera vista podríamos concluir que tenemos un modelo excelente. Pero inmediatamente aparece una pregunta: ¿95 % de aciertos sobre qué datos? Si evaluamos el modelo utilizando exactamente los mismos datos que utilizó para aprender, ese porcentaje puede resultar engañoso.

Un modelo de Machine Learning no tiene como objetivo demostrar que puede reproducir los datos que ya conoce. Su verdadera utilidad aparece cuando recibe una observación que no estuvo presente durante el aprendizaje y consigue producir una predicción razonable. Por eso, a partir de esta clase, debemos comenzar a pensar en términos de generalización.

7.1. Predicción, valor real y error

Todo proceso de evaluación comienza comparando lo que el modelo predijo con lo que realmente ocurrió.

Supongamos que nuestro sistema analiza un cliente y determina que realizará una compra. Posteriormente sabemos que efectivamente compró. La predicción fue correcta.

En otro caso, el modelo puede predecir que el cliente comprará, pero finalmente no realiza ninguna compra. La predicción fue incorrecta.

En términos generales tenemos:

prediccio´n↔valor realpredicción \quad \leftrightarrow \quad valor\ real

La diferencia entre ambos constituye el punto de partida para analizar el error del modelo.

En regresión, el error puede tener una magnitud concreta. Si el modelo predice que un cliente gastará $200.000 y finalmente gasta $180.000, existe una diferencia de $20.000. En clasificación, en cambio, normalmente comenzamos preguntando si la predicción pertenece a la clase correcta.

Pero incluso en clasificación la situación es más compleja de lo que parece. No todos los errores tienen necesariamente la misma importancia.

Imaginemos un sistema que intenta detectar transacciones fraudulentas. Si el modelo clasifica una transacción legítima como fraude, tenemos un problema. Pero si clasifica una transacción fraudulenta como legítima, también tenemos un problema, y posiblemente las consecuencias sean mucho más importantes.

Por eso no alcanza con contar únicamente cuántas predicciones fueron correctas.

7.2. Función de pérdida y desempeño

Durante el entrenamiento, los modelos necesitan algún mecanismo que permita determinar qué tan buenas o malas son sus predicciones. Para ello aparecen las funciones de pérdida o funciones de costo.

La idea general es sencilla: una predicción que se aleja mucho del resultado esperado debería producir una penalización mayor que una predicción cercana.

En regresión, por ejemplo, podemos calcular diferencias entre valores reales y predichos y construir una función que represente el error global.

En clasificación, podemos utilizar funciones de pérdida apropiadas para evaluar qué tan lejos se encuentran las predicciones de los resultados esperados.

Es importante diferenciar dos ideas que muchas veces se confunden. La función de pérdida se utiliza dentro del proceso de aprendizaje para orientar el entrenamiento, mientras que las métricas nos ayudan a analizar el desempeño del modelo.

No necesariamente debemos utilizar la misma medida para entrenar y para comunicar o interpretar el desempeño final.

7.3. La matriz de confusión

Para comprender correctamente la evaluación de un clasificador necesitamos abandonar momentáneamente la idea de "acertó o falló" y observar qué tipo de acierto o error produjo.

Supongamos nuevamente que queremos detectar si una operación es fraudulenta.

Tenemos dos clases:

Fraude
No fraude

El modelo realiza predicciones y posteriormente podemos comparar esas predicciones con la realidad.

A partir de esa comparación aparecen cuatro posibilidades fundamentales.

Un verdadero positivo, o TP, ocurre cuando el modelo predice positivo y efectivamente el caso es positivo.

Un verdadero negativo, o TN, ocurre cuando el modelo predice negativo y efectivamente el caso es negativo.

Un falso positivo, o FP, ocurre cuando el modelo predice positivo pero el caso realmente era negativo.

Un falso negativo, o FN, ocurre cuando el modelo predice negativo pero el caso realmente era positivo.

Podemos representarlo conceptualmente de la siguiente manera:

	Real positivo	Real negativo
Predicción positiva	TP	FP
Predicción negativa	FN	TN

Esta tabla constituye la matriz de confusión.

Su importancia es enorme porque nos permite dejar de observar solamente el porcentaje global de aciertos y comenzar a estudiar la naturaleza de los errores.

7.4. Un ejemplo sencillo

Supongamos que tenemos 100 operaciones y nuestro modelo produjo los siguientes resultados:

	Real fraude	Real legítima
Predijo fraude	18	7
Predijo legítima	5	70

Tenemos entonces:

TP=18TP=18 FP=7FP=7 FN=5FN=5 TN=70TN=70

El modelo acertó 18 fraudes y 70 operaciones legítimas.

En total:

18+70=8818+70=88

predicciones correctas.

Por lo tanto, la accuracy será:

Accuracy=TP+TNTP+TN+FP+FNAccuracy=\frac{TP+TN}{TP+TN+FP+FN}
Accuracy=18+70100=0.88Accuracy=\frac{18+70}{100}=0.88

Es decir, 88 %.

Hasta aquí podríamos decir que el modelo tiene un 88 % de accuracy.

Pero todavía no sabemos si ese resultado es suficientemente bueno para nuestro problema.

7.5. Accuracy

La accuracy representa la proporción de predicciones correctas respecto del total de observaciones.

Accuracy=TP+TNTP+TN+FP+FNAccuracy= \frac{TP+TN} {TP+TN+FP+FN}

Es una métrica intuitiva y útil en muchos problemas.

Sin embargo, tiene una limitación importante: puede ocultar problemas cuando las clases están desbalanceadas.

Imaginemos un sistema de detección de fraude donde de 10.000 operaciones solamente 100 son fraudulentas.

Supongamos que un modelo predice:

Todas las operaciones → legítimas

El modelo acertaría 9.900 de las 10.000 operaciones.

Su accuracy sería:

99%99\%

A primera vista parece extraordinario.

Pero el modelo no detectó ningún fraude.

Para un sistema cuyo objetivo principal es detectar operaciones fraudulentas, ese modelo sería prácticamente inútil.

Esto demuestra por qué una métrica aparentemente excelente puede esconder un comportamiento inadecuado.

7.6. Precision

La precision responde a una pregunta diferente:

> De todos los casos que el modelo clasificó como positivos, ¿cuántos realmente eran positivos?

Su expresión es:

Precision=TPTP+FPPrecision= \frac{TP}{TP+FP}

En nuestro ejemplo:

Precision=1818+7Precision= \frac{18}{18+7} Precision=0.72Precision=0.72

La precision es entonces 72 %.

Interpretémosla en términos del problema.

De todas las operaciones que el modelo marcó como fraudulentas, aproximadamente el 72 % realmente eran fraudulentas.

Esta métrica resulta especialmente importante cuando los falsos positivos tienen un costo elevado.

Por ejemplo, si un sistema bloquea automáticamente tarjetas de crédito ante una sospecha de fraude, demasiados falsos positivos pueden provocar que clientes legítimos tengan operaciones rechazadas.

7.7. Recall

El recall, también denominado sensibilidad, responde a otra pregunta:

> De todos los casos positivos que realmente existían, ¿cuántos consiguió detectar el modelo?

La fórmula es:

Recall=TPTP+FNRecall= \frac{TP}{TP+FN}

En nuestro ejemplo:

Recall=1818+5Recall= \frac{18}{18+5} Recall≈0.783Recall\approx0.783

Tenemos aproximadamente un 78,3 % de recall.

Esto significa que el modelo detectó aproximadamente el 78 % de los fraudes existentes.

El resto fueron falsos negativos.

Aquí aparece una diferencia fundamental entre precision y recall.

La precision se concentra en la calidad de los positivos que el modelo encontró.

El recall se concentra en la cantidad de positivos reales que consiguió encontrar.

7.8. Precision y Recall representan problemas diferentes

Supongamos que desarrollamos un sistema para identificar contenido potencialmente peligroso.

Podemos construir un modelo muy conservador que solamente marque como positivo aquellos casos de los que está prácticamente seguro.

Probablemente tendrá una precision elevada, porque casi todo lo que marque será realmente positivo.

Pero puede dejar pasar muchos casos.

En ese escenario tendremos un recall menor.

Por el contrario, podemos construir un modelo mucho más sensible que marque una gran cantidad de casos como positivos. Podría detectar casi todos los casos reales, aumentando el recall, pero al costo de generar muchos falsos positivos.

Por eso no existe una regla universal que diga:

> "Siempre debemos maximizar precision."

o:

> "Siempre debemos maximizar recall."

La métrica adecuada depende del problema y del costo de cada tipo de error.

7.9. F1-Score

Cuando necesitamos considerar conjuntamente precision y recall podemos utilizar el F1-score.

Se define como la media armónica entre ambas:

F1=2 · Precision · RecallPrecision+RecallF1= 2\cdot \frac{Precision\cdot Recall}{Precision+Recall}

La media armónica hace que un valor muy bajo de precision o recall penalice considerablemente el resultado final.

Esto es importante porque un modelo no puede obtener un F1 elevado simplemente teniendo una de las dos métricas muy alta mientras la otra es muy baja.

El F1-score resulta especialmente útil cuando necesitamos un equilibrio entre detectar correctamente los positivos y evitar clasificar incorrectamente demasiados negativos como positivos.

7.10. ¿Cuál métrica debemos utilizar?

No existe una respuesta universal.

Supongamos tres problemas diferentes.

En un sistema donde queremos detectar la mayor cantidad posible de casos peligrosos, puede ser especialmente importante minimizar los falsos negativos. En ese caso el recall puede tener una importancia prioritaria.

En un sistema donde una falsa alarma genera un costo elevado, podemos necesitar prestar especial atención a la precision.

En situaciones donde necesitamos equilibrar ambos objetivos, F1 puede ser una alternativa adecuada.

Accuracy puede resultar perfectamente válida cuando las clases están relativamente equilibradas y los diferentes errores tienen una importancia similar.

La elección de la métrica debe entonces comenzar por una pregunta mucho más general:

> ¿Qué significa equivocarse en este problema?

Esta pregunta es más importante que memorizar cualquier fórmula.

7.11. Evaluación de modelos de regresión

Hasta ahora estuvimos analizando clasificación. Pero la regresión necesita otro conjunto de métricas porque la salida del modelo es numérica.

Una de las medidas más intuitivas es el MAE, Mean Absolute Error.

MAE=1n∑i=1n∣yi−y^i∣MAE= \frac{1}{n} \sum_{i=1}^{n} |y_i-\hat{y}_i|

El MAE calcula la diferencia absoluta promedio entre los valores reales y las predicciones.

Supongamos que tenemos estos errores absolutos:

10
20
30
40

El MAE será:

MAE=10+20+30+404=25MAE=\frac{10+20+30+40}{4}=25

Podemos interpretarlo directamente en las unidades de la variable objetivo.

Si estamos prediciendo precios en pesos, un MAE de 25.000 significa que, en promedio, las predicciones se alejan aproximadamente $25.000 del valor real.

7.12. MSE y RMSE

Otra medida frecuente es el MSE, Mean Squared Error.

MSE=1n∑i=1n(yi−y^i)2MSE= \frac{1}{n} \sum_{i=1}^{n} (y_i-\hat{y}_i)^2

Aquí los errores se elevan al cuadrado.

Esto tiene una consecuencia importante: los errores grandes reciben una penalización mucho mayor.

Si un error es 2:

22=42^2=4

pero si el error es 10:

102=10010^2=100

Por eso MSE es especialmente sensible a errores grandes.

El RMSE, Root Mean Squared Error, consiste en obtener la raíz cuadrada del MSE:

RMSE=MSERMSE=\sqrt{MSE}

Esto permite volver a las unidades originales de la variable objetivo.

MAE y RMSE pueden contar historias diferentes sobre un modelo, especialmente cuando existen algunos errores particularmente grandes.

7.13. R²

Otra métrica habitual para regresión es R2R^2, denominada coeficiente de determinación.

Su interpretación requiere algo más de cuidado que simplemente decir que "cuanto más alto mejor".

De manera general, permite evaluar qué tan bien el modelo explica la variabilidad observada en los datos en relación con un modelo de referencia basado en la media.

Un R2R^2 cercano a 1 indica un ajuste muy elevado bajo esta medida.

Un valor cercano a 0 indica que el modelo no mejora demasiado respecto de esa referencia.

Incluso pueden aparecer valores negativos en determinadas situaciones, particularmente cuando el modelo tiene un desempeño peor que la predicción basada simplemente en la media.

Por eso tampoco debemos interpretar R2R^2 de manera aislada.

7.14. El problema fundamental: ¿con qué datos evaluamos?

Volvamos ahora a una situación fundamental.

Tenemos 5.000 registros en nuestro dataset.

Podríamos utilizar todos para entrenar el modelo.

Después podríamos calcular su accuracy utilizando esos mismos 5.000 registros.

Pero estaríamos haciendo algo problemático.

El modelo acaba de aprender utilizando esos datos y luego le estamos preguntando qué tan bien reproduce esos mismos datos.

Sería parecido a entregar a un estudiante exactamente las preguntas del examen antes de rendir y después evaluar su rendimiento con esas mismas preguntas.

El resultado no nos permitiría saber si realmente aprendió.

Necesitamos separar los datos.

7.15. Train, validation y test

Una división conceptual habitual es:

Datos
↓
Train
Validation
Test

El conjunto train se utiliza para entrenar el modelo.

El conjunto validation se utiliza para tomar decisiones durante el desarrollo, comparar configuraciones y ajustar hiperparámetros.

El conjunto test debe reservarse para la evaluación final.

La idea fundamental es que el test debe representar una instancia que el modelo no utilizó para tomar decisiones durante el proceso de construcción.

Podríamos imaginar:

5000 registros

Train → 3500
Validation → 750
Test → 750

Los porcentajes concretos pueden variar según el problema y el tamaño del dataset. Lo importante no es memorizar una división exacta, sino comprender la función de cada conjunto.

7.16. ¿Por qué no debemos usar el test durante el ajuste?

Supongamos que entrenamos cinco modelos.

Evaluamos todos utilizando el conjunto de test.

Observamos que el modelo número 4 obtiene el mejor resultado y entonces modificamos algunos hiperparámetros para intentar mejorarlo.

Volvemos a mirar el test.

Después probamos otra configuración y volvemos a mirar el test.

Aunque no estemos modificando directamente los datos del test, estamos utilizando indirectamente esa información para tomar decisiones.

El test deja entonces de ser una evaluación verdaderamente independiente.

Por eso el conjunto de test debe utilizarse como una especie de evaluación final.

Durante el desarrollo trabajamos principalmente con train y validation.

Cuando finalmente seleccionamos nuestro modelo, podemos realizar la evaluación final sobre test.

7.17. Validación cruzada

Cuando tenemos un dataset limitado, una única partición entre entrenamiento y validación puede depender demasiado de qué observaciones quedaron en cada conjunto.

La validación cruzada permite reducir este problema.

En K-Fold Cross Validation dividimos los datos en KK partes.

Por ejemplo, con cinco particiones:

Fold 1
Fold 2
Fold 3
Fold 4
Fold 5

En una primera iteración utilizamos una partición para validar y las otras cuatro para entrenar.

Después cambiamos la partición utilizada para validar.

Repetimos el proceso hasta que cada fold haya sido utilizado como validación.

Finalmente podemos calcular el promedio de los resultados obtenidos.

Conceptualmente:

Iteración 1 → Train Train Train Train | Val
Iteración 2 → Train Train Train | Val | Train
Iteración 3 → Train Train | Val | Train Train
...

La ventaja es que cada observación tiene la posibilidad de formar parte del conjunto de validación durante alguna iteración.

Esto proporciona una evaluación más robusta que depender exclusivamente de una única partición.

7.18. Limitaciones de la validación cruzada

La validación cruzada no es una solución mágica.

Su principal costo es computacional.

Si realizamos cinco entrenamientos para evaluar una configuración, necesitamos entrenar el modelo varias veces.

Además, no reemplaza automáticamente al conjunto de test cuando queremos realizar una evaluación final independiente.

También debemos tener cuidado con problemas donde existe dependencia temporal o estructura particular en los datos. No siempre podemos dividir aleatoriamente las observaciones sin considerar la naturaleza del problema.

Esto nos recuerda nuevamente una idea importante: las técnicas de Machine Learning deben aplicarse teniendo en cuenta el contexto de los datos.

7.19. Generalización

Llegamos ahora a uno de los conceptos centrales de la clase.

Un modelo generaliza cuando puede utilizar lo aprendido a partir de los datos de entrenamiento para realizar predicciones razonables sobre datos nuevos.

Supongamos que entrenamos un modelo con información de clientes de enero a octubre y posteriormente lo utilizamos con clientes de noviembre.

Si mantiene un buen desempeño, podemos decir que está generalizando razonablemente bien.

El verdadero objetivo del aprendizaje no es memorizar enero a octubre.

El objetivo es aprender patrones que continúen siendo útiles cuando aparezcan observaciones que el modelo nunca había visto.

Esta es la razón por la cual la evaluación sobre datos no utilizados durante el entrenamiento es tan importante.

7.20. Error de entrenamiento y error de validación

Podemos representar el comportamiento de un modelo utilizando dos errores:

ErrortrainError_{train}

y

ErrorvalidationError_{validation}

Supongamos que obtenemos:

Train → 5 %
Validation → 7 %

La diferencia es relativamente pequeña.

Ahora imaginemos:

Train → 1 %
Validation → 35 %

Aquí tenemos una diferencia enorme.

El modelo funciona extraordinariamente bien sobre los datos de entrenamiento, pero su comportamiento empeora considerablemente sobre datos que no utilizó para aprender.

Esto es una señal clara de problemas de generalización.

7.21. Underfitting

El underfitting, o subajuste, ocurre cuando el modelo es demasiado simple para capturar adecuadamente la estructura existente en los datos.

Podemos observar algo como:

Train → desempeño bajo
Validation → desempeño bajo

El modelo no logra aprender suficientemente bien ni siquiera los datos de entrenamiento.

Imaginemos que intentamos representar una relación claramente curva utilizando una línea recta demasiado restrictiva.

El modelo simplemente no tiene suficiente capacidad para representar la relación.

El problema no necesariamente consiste en que necesitemos "más datos". Puede estar relacionado con la representación de las características, la elección del modelo, sus hiperparámetros o su nivel de complejidad.

7.22. Overfitting

El overfitting, o sobreajuste, aparece cuando el modelo se adapta excesivamente a los datos de entrenamiento.

Una situación típica sería:

Train → 99 %
Validation → 72 %

El modelo parece excelente durante el entrenamiento, pero pierde desempeño sobre datos nuevos.

Una forma intuitiva de entenderlo es pensar que el modelo no solamente aprendió patrones generales, sino también detalles particulares del conjunto de entrenamiento, incluyendo posiblemente ruido.

Esto no significa literalmente que el modelo "memorice" todos los datos en todos los casos, pero la idea de memorización resulta útil como intuición inicial.

El problema fundamental es la falta de generalización.

7.23. Complejidad y generalización

Podemos imaginar la complejidad del modelo sobre un eje horizontal:

Baja complejidad ---------- Alta complejidad
↑ ↑
Underfitting Overfitting

En una zona intermedia podemos encontrar configuraciones capaces de aprender suficientemente bien sin adaptarse excesivamente al conjunto de entrenamiento.

Esta relación constituye uno de los problemas centrales de Machine Learning.

No buscamos el modelo más simple posible.

Tampoco buscamos el modelo más complejo posible.

Buscamos una configuración que permita aprender los patrones relevantes y generalizar adecuadamente.

7.24. Sesgo y varianza

Para comprender mejor esta situación aparece el concepto de bias-variance trade-off, o compromiso entre sesgo y varianza.

El bias representa, de manera simplificada, el error asociado a utilizar un modelo con supuestos demasiado restrictivos.

Un modelo excesivamente simple puede presentar un sesgo elevado porque no tiene capacidad suficiente para representar la relación real.

La variance, por otra parte, está relacionada con la sensibilidad del modelo respecto de los datos de entrenamiento.

Un modelo demasiado complejo puede cambiar significativamente cuando cambia el conjunto de entrenamiento.

De manera intuitiva:

Mucho bias
↓
Modelo demasiado simple
↓
Underfitting

Mucho variance
↓
Modelo demasiado sensible
↓
Overfitting

El objetivo es encontrar un equilibrio.

7.25. Error irreducible

Incluso si construimos un modelo muy sofisticado, no necesariamente podremos eliminar todo el error.

Los datos reales contienen ruido, información incompleta, mediciones imperfectas y factores que no están disponibles para el modelo.

Existe entonces una parte del error que no podemos eliminar simplemente aumentando la complejidad del algoritmo.

A esta componente se la suele denominar error irreducible.

Esto es importante porque evita una expectativa equivocada: no existe un modelo perfecto simplemente porque dispongamos de suficiente capacidad computacional.

El objetivo es construir modelos suficientemente buenos para el problema real y suficientemente robustos para generalizar.

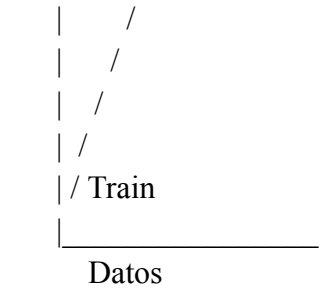
7.26. Curvas de aprendizaje

Una herramienta útil para analizar el comportamiento de un modelo son las curvas de aprendizaje.

Podemos estudiar cómo evolucionan determinadas métricas cuando aumenta la cantidad de datos utilizados para entrenar.

Conceptualmente podemos observar:

Desempeño
|
| Validation



Las curvas pueden ayudarnos a identificar patrones relacionados con underfitting y overfitting.

Por ejemplo, si el modelo obtiene un desempeño excelente en entrenamiento pero existe una diferencia importante y persistente respecto de validation, podemos sospechar sobreajuste.

Si ambos desempeños son bajos y permanecen bajos, podemos sospechar que el modelo no tiene suficiente capacidad o que existe otro problema en la representación de los datos.

Las curvas no sustituyen al análisis, pero ayudan a visualizarlo.

7.27. Curvas de validación

También podemos analizar el comportamiento del modelo frente a diferentes valores de un hiperparámetro.

Supongamos que entrenamos un árbol con diferentes profundidades:

Profundidad
3
5
7
10
15
20

Para cada configuración podemos calcular el desempeño de entrenamiento y validación.

Podríamos observar:

Profundidad	Train	Validation
3	0,78	0,75
5	0,84	0,81
7	0,91	0,84

10	0,97	0,79
15	1,00	0,72

La profundidad 15 produce un entrenamiento perfecto, pero validation empeora considerablemente.

En cambio, una profundidad intermedia presenta un mejor equilibrio.

Esto muestra de manera experimental que aumentar la complejidad no garantiza un mejor modelo.

7.28. Evaluar los modelos de la clase anterior

Podemos volver ahora a los modelos que estudiamos anteriormente.

Supongamos que entrenamos KNN, un árbol de decisión y regresión logística para predecir compra/no compra.

La evaluación podría comenzar con:

Modelo	Train	Validation
KNN	0,86	0,82
Árbol	0,99	0,76
Logística	0,83	0,81

Si observamos solamente train, podríamos elegir inmediatamente el árbol.

Pero si observamos validation, la situación cambia.

El árbol presenta una diferencia considerable entre entrenamiento y validación.

Esto nos proporciona una señal de posible overfitting.

KNN y regresión logística presentan comportamientos más cercanos entre entrenamiento y validación.

Pero todavía falta una parte fundamental: ¿qué tipos de errores están cometiendo?

Podría ocurrir que KNN tenga una accuracy ligeramente inferior pero un recall considerablemente mayor.

En ese caso, dependiendo del objetivo del sistema, KNN podría ser preferible.

Aquí se completa el razonamiento que comenzamos al principio de la clase.

No debemos preguntar simplemente:

> "¿Cuál tiene mayor accuracy?"

Debemos preguntar:

> "¿Cuál resuelve mejor nuestro problema y generaliza adecuadamente?"

7.29. El flujo de evaluación

A esta altura podemos integrar todo lo aprendido.

Tenemos un problema.

Disponemos de datos.

Separamos features y target.

Dividimos los datos en train, validation y test.

Entrenamos diferentes modelos.

Realizamos predicciones.

Calculamos métricas.

Analizamos errores.

Modificamos hiperparámetros.

Volvemos a entrenar.

Comparamos resultados.

Seleccionamos una configuración.

Finalmente utilizamos el conjunto de test para realizar una evaluación independiente.

Podemos visualizarlo como:

Datos
↓
Train / Validation / Test
↓
Entrenamiento
↓

Predicción
↓
Métricas
↓
Análisis de errores
↓
Ajuste de hiperparámetros
↓
Nuevo experimento
↓
Selección del modelo
↓
Evaluación final con Test

Este flujo será reutilizado prácticamente durante toda la materia.

7.30. Una cuestión fundamental: un modelo "bueno" depende del problema

Supongamos que tenemos dos modelos.

El primero obtiene:

Accuracy = 95 %
Recall = 60 %

El segundo:

Accuracy = 92 %
Recall = 90 %

¿Cuál es mejor?

No podemos responder sin conocer el problema.

Si estamos intentando detectar fraudes y perder un fraude resulta muy costoso, probablemente el segundo modelo sea más interesante.

Si el sistema genera acciones automáticas muy costosas ante cada falso positivo, quizás necesitemos prestar mayor atención a precision.

La evaluación no consiste entonces en encontrar el número más grande.

Consiste en relacionar las métricas con el objetivo real del sistema.

Esta idea es fundamental para evitar uno de los errores más frecuentes al comenzar con Machine Learning: optimizar una métrica sin preguntarse si esa métrica representa realmente lo que importa.

7.31. Cierre de la clase

Llegamos así a uno de los puntos centrales del recorrido de Machine Learning. En la clase anterior aprendimos a construir modelos; en esta clase aprendimos que construir un modelo es solamente una parte del problema.

Un modelo debe ser evaluado sobre datos que permitan determinar si realmente aprendió patrones útiles. Para clasificación vimos que la matriz de confusión permite distinguir verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos. A partir de ellos construimos accuracy, precision, recall y F1-score, comprendiendo que cada métrica responde una pregunta diferente.

Para regresión introdujimos MAE, MSE, RMSE y R^2, observando que también aquí la elección de la métrica depende de aquello que queremos medir.

Después analizamos la separación entre train, validation y test y comprendimos por qué no debemos utilizar el conjunto de prueba para ajustar continuamente nuestro modelo. Introdujimos también la validación cruzada como una estrategia para obtener evaluaciones más robustas cuando disponemos de una cantidad limitada de datos.

Finalmente llegamos al concepto que conecta toda la clase: generalización. Un modelo no es bueno porque memoriza los datos de entrenamiento; es bueno cuando logra utilizar lo aprendido para realizar predicciones razonables sobre datos que nunca vio.

El underfitting aparece cuando el modelo resulta demasiado simple y no consigue aprender adecuadamente. El overfitting aparece cuando el modelo se adapta excesivamente al entrenamiento y pierde capacidad de generalización. El compromiso entre bias y variance nos ayuda a comprender por qué existe una zona intermedia en la que debemos buscar el equilibrio adecuado entre capacidad y generalización.

En la próxima clase podremos avanzar un paso más. Ya sabemos entrenar, evaluar y detectar problemas de generalización. Ahora podemos preguntarnos qué podemos hacer para mejorar deliberadamente un modelo. Allí entrarán en juego SVM, los ensambles, Random Forest, Boosting y, especialmente, los hiperparámetros y la experimentación sistemática.